x86-64 OS Development Roadmap

Oct 27, 2026 · @momo

Goal and ground rules

The goal is a 64-bit kernel in C that boots in QEMU, multitasks, runs user programs from disk and gives you a shell. At about 8 hours a week, expect 5 to 8 months. Language is C with a little NASM assembly; Rust can replace C later if you prefer.

Definition of done for the first version:

- Boots from an ISO in QEMU and in VirtualBox
- Preemptive scheduler running several processes
- User mode (ring 3) with system calls
- Loads ELF programs from a filesystem
- A shell that can list files and launch programs

Rules that keep the project alive:

1. Always keep the kernel bootable. Commit after every working milestone and tag it (`v0.1-boot`, `v0.2-idt`, and so on).
2. Serial logging first. Every subsystem prints its state to the serial port so you can debug without a debugger.
3. One concept per week. Read the theory, then code it, then write 5 lines of notes on what broke.
4. Don't skip ahead. Paging bugs look like scheduler bugs, so each phase must be solid before the next.
5. Test in QEMU daily, in VirtualBox at each phase end, on real hardware only at the very end.

Phase 0: Setup (2 to 4 days)

A working build-and-run loop comes before any kernel code, because every later phase depends on it. Develop on Linux (native, WSL or a VM) and use QEMU as your main test machine.

Install:

- `x86_64-elf-gcc` and `binutils` (cross-compiler, built from source or from your distro's packages)
- `nasm`, `make`, `xorriso`, `git`
- `qemu-system-x86_64` and `gdb`
- Limine bootloader (use the binary release branch)

Suggested repo layout:

```
myos/
  kernel/        # C sources: main.c, drivers/, mm/, sched/, fs/
  arch/x86_64/   # assembly: gdt, idt stubs, context switch
  user/          # user programs, later
  limine.conf    # bootloader config
  linker.ld      # kernel linker script
  Makefile       # build, iso, run, debug targets
```

Daily commands to set up as Makefile targets:

- `make run`: `qemu-system-x86_64 -cdrom os.iso -serial stdio -no-reboot -no-shutdown -m 512M`
- `make debug`: same plus `-s -S`, then `gdb kernel.elf` and `target remote :1234`
- `make trace`: add `-d int,cpu_reset -D qemu.log` to see why you triple-faulted

Exit gate: your kernel compiles, boots from the ISO, and `make run` shows the bootloader handing off to your code (even if it only halts).

Phase 1: Boot and output (1 week)

Your kernel prints text two ways: the serial port for debugging and the framebuffer for the screen. Serial comes first because it works even when everything else is broken.

Build:

1. Linker script placing the kernel in the higher half and a Limine request block (framebuffer, memory map, HHDM offset)
2. `kmain()` entry point that halts in a `hlt` loop
3. Serial driver for COM1 (port `0x3F8`): init, `serial_putc`, `serial_puts`
4. `kprintf` with `%s %d %x %p %c`, writing to serial first
5. Framebuffer text output with a bitmap font (an 8x16 PSF font is enough), scrolling and a cursor

6. A `panic()` that prints a message, registers and halts

Concepts to learn: how Limine leaves the CPU (long mode, paging on, interrupts off), the higher-half kernel idea, ports and `in` / `out`, freestanding C (no libc, `-ffreestanding -mno-red-zone`).

Exit gate: `kprintf("hello %d\n", 42)` appears on both the QEMU window and the terminal.

Tag: `v0.1-boot`

Phase 2: CPU setup and interrupts (1 to 2 weeks)

This phase turns CPU faults from silent reboots into readable error messages, which makes every later bug cheaper to find. Expect your first triple faults here.

Build:

1. Your own GDT with kernel code/data, user code/data and a TSS entry; reload segment registers
2. IDT with 256 entries and assembly stubs that save registers into a common `interrupt_frame` struct
3. Exception handlers for divide error, invalid opcode, general protection fault and page fault; the page fault handler prints `CR2` and the error code
4. Remap and mask the legacy PIC, then move to the Local APIC and I/O APIC (ACPI MADT parsing is needed for this)
5. Deliberately trigger each exception to prove the handlers work

Concepts to learn: segmentation in long mode, privilege rings, the interrupt stack table (IST) for double faults, why the TSS still matters in 64-bit mode.

Exit gate: a null pointer dereference prints `PAGE FAULT at 0x0` with a register dump instead of rebooting.

Tag: `v0.2-interrupts`

Phase 3: Memory management (2 to 3 weeks)

Memory management is the hardest foundation phase, and every later subsystem allocates from it. Build it in three layers, testing each before starting the next.

Layer 1, physical memory manager (PMM):

1. Read the Limine memory map and mark usable 4 KiB frames
2. Bitmap allocator with `pmm_alloc_frame()` and `pmm_free_frame()`
3. Print total, used and free memory at boot as a sanity check

Layer 2, virtual memory (VMM):

1. Learn the 4-level page table: PML4, PDPT, PD, PT, with the flags present, writable, user, no-execute
2. `vmm_map(virt, phys, flags)`, `vmm_unmap()` and `vmm_translate()`
3. Use the higher-half direct map (HHDM) to touch physical frames
4. Create your own kernel page tables and load `CR3`; remember to invalidate with `invlpg`
5. A page fault handler that can tell a bad access from a valid one

Layer 3, kernel heap:

1. A simple `kmalloc` / `kfree` first (free list or bump allocator), later a slab allocator for fixed-size objects
2. Heap grows by mapping new pages on demand
3. Stress test: allocate and free thousands of random-size blocks and check for corruption

Exit gate: you can allocate 100 MB through `kmalloc`, free it, and allocate again without leaks or faults.

Tag: `v0.3-memory`

Phase 4: Time, input and multitasking (2 to 3 weeks)

This is the phase where the project starts to feel like an operating system: the CPU is shared between several things at once and you can type into it.

Build:

1. Timer tick from the LAPIC timer (calibrate it against the PIT or HPET), exposing `uptime_ms()` and `sleep_ms()`
2. PS/2 keyboard driver: scancode set 1 to ASCII, a key event queue, shift and caps handling
3. Task structure holding saved registers, kernel stack, state and a pid
4. Context switch in assembly: save callee-saved registers, swap stack pointers

5. Round-robin scheduler driven by the timer tick, with `yield()`, `sleep()` and blocked/ready states
6. Spinlocks (with interrupts disabled while held) and a simple semaphore or mutex
7. A kernel thread demo: three threads printing different letters while the keyboard stays responsive

Concepts to learn: preemption vs cooperative scheduling, race conditions and why interrupts and locks interact, stack layout during a switch. OSTEP chapters on scheduling and concurrency fit here.

Exit gate: three kernel threads run concurrently for 10 minutes without a crash, and typed keys reach a kernel buffer.

Tag: `v0.4-multitasking`

Phase 5: User mode and system calls (3 to 4 weeks)

User mode is what separates a kernel from a program that happens to run on bare metal: programs run unprivileged, in their own address space, and ask the kernel for everything through system calls.

Build:

1. Per-process address space: a new PML4 for each process that shares the kernel half
2. Enter ring 3 with `iretq` to a user stack and user entry point
3. `syscall` / `sysret` setup (MSRs `STAR`, `LSTAR`, `SFMASK`) and a syscall dispatch table
4. First syscalls: `write`, `exit`, `getpid`, `yield`, then `sleep`, `brk` / `mmap`
5. Validate every user pointer before the kernel reads or writes it
6. ELF64 loader: parse program headers, map segments, set up the stack with `argc` / `argv`
7. `fork` and `exec` (or a simpler `spawn`), plus `wait` and `exit` for process lifetime
8. A tiny userspace library (`libc` subset): `printf`, `malloc`, string functions, syscall wrappers

Concepts to learn: privilege transitions, why kernel and user stacks are separate, copy-on-write (optional but rewarding), the System V x86-64 calling convention.

Exit gate: a user program written in C, compiled separately, loads as an ELF, prints text through `write`, and a crashing user program is killed without taking down the kernel.

Tag: `v0.5-usermode`

Phase 6: Filesystem, storage and shell (3 to 5 weeks)

Programs need somewhere to live and a way for you to launch them. Start with a ramdisk so you can test the VFS without a disk driver, then add real storage.

Build:

1. VFS layer: `open`, `read`, `write`, `close`, `lseek`, `stat`, `readdir`, with a per-process file descriptor table
2. Initial ramdisk (a tar archive loaded as a Limine module) mounted at `/`; `tmpfs` for `/tmp`
3. Device files: `/dev/console` and `/dev/null`, so stdin, stdout and stderr work as file descriptors
4. Storage driver: AHCI (SATA) or virtio-blk, which is easier in QEMU
5. A real filesystem: read-only FAT32 first, then read/write; ext2 is a good later step
6. Block cache so repeated reads don't hit the disk
7. Pipes and simple redirection
8. A shell in userspace: prompt, parse a line, `fork` + `exec`, built-ins `cd`, `ls`, `cat`, `echo`, `clear`, `help`
9. A handful of utilities as separate programs (`ls`, `cat`, `echo`, `ps`, `mem`)

Concepts to learn: inodes, directory entries, block I/O and DMA, why a cache is needed, how POSIX file descriptors map onto kernel objects.

Exit gate: boot to a prompt, run `ls`, `cat hello.txt` and a program you compiled yesterday, all from files on a disk image.

Tag: `v1.0-shell`. This is your first complete OS.

Phase 7: Stretch goals (pick by interest)

After v1.0 the order is yours. Each item below stands alone, so choose by what you want to learn, not by difficulty alone.

Goal	What you build	Why it's worth it	Rough effort
SMP (multicore)	Wake the other CPUs via LAPIC IPIs, per-CPU data, a scheduler with per-core run queues, TLB shootdowns	Deepest concurrency lesson there is; ties directly to distributed systems	3 to 6 weeks
Networking	virtio-net or e1000 driver, Ethernet, ARP, IP, ICMP (ping), UDP, then a small TCP and a socket API	Direct overlap with your network and distributed systems master's	4 to 8 weeks
Port real software	A fuller libc (or port `newlib` / `musl`), then run `busybox`, a text editor, maybe `lua`	Proves your syscall interface is complete	3 to 6 weeks
Graphics and GUI	Framebuffer compositor, mouse driver, windows, a tiny widget toolkit	The most visible result; great for demos and content	4 to 8 weeks
USB	xHCI controller, HID keyboard and mouse, mass storage	Needed to run on real modern hardware	6+ weeks
Security hardening	SMEP/SMAP, NX, ASLR, stack canaries, per-process limits	Fits a systems-security mindset	2 to 4 weeks
Own bootloader	Real mode to long mode yourself, then load the kernel from FAT	Closes the one piece you skipped with Limine	2 to 4 weeks
Rewrite a subsystem in Rust	For example the allocator or a driver, with C interop	Compare both languages on the same problem	2 to 3 weeks

My suggestion: networking first (it overlaps your studies), then SMP, then porting software.

Reading list and weekly rhythm

Read the theory the week before you code it. The week counts below assume about 8 hours a week and are approximate; slow down on phases 3 and 5, which are where most hobby OS projects stall.

Phase	Weeks (cumulative)	Read alongside
0. Setup	0.5	OSDev wiki: Limine Bare Bones, GCC Cross-Compiler
1. Boot and output	1.5	OSDev wiki: Serial Ports, Printing to Screen; Phil Opp's blog (first posts)
2. CPU and interrupts	3.5	OSDev wiki: GDT, IDT, APIC; Intel SDM volume 3, chapters on interrupts
3. Memory	6.5	OSTEP: virtual memory chapters; Intel SDM volume 3, paging
4. Multitasking	9.5	OSTEP: scheduling and concurrency; OSDev wiki: Context Switching
5. User mode	13.5	OSDev wiki: Getting to Ring 3, System Calls; the ELF specification
6. Filesystem and shell	18.5	OSTEP: persistence chapters; OSDev wiki: FAT, AHCI
7. Stretch goals	open	Pick per goal

Core references:

- OSDev wiki: the practical reference; check each page for 64-bit notes
- Operating Systems: Three Easy Pieces: free, the theory behind every phase
- Writing an OS in Rust: excellent explanations even if you code in C
- Limine bootloader: protocol and bare-bones template
- Intel SDM volume 3 and AMD64 manual volume 2: the authority on paging, GDT, IDT and MSRs
- xv6 source and book (MIT): a small, readable Unix-like kernel to compare with yours

Weekly rhythm (about 8 hours):

- 2 hours reading and planning the week's milestone
- 4 hours coding
- 1 hour debugging with GDB and QEMU traces
- 1 hour of notes, commit and tag

Where to get unstuck: the OSDev forum and Discord, the `r/osdev` subreddit, and reading how xv6 or SerenityOS solved the same problem. Keep a `NOTES.md` of every bug and its cause; by phase 4 it becomes your best reference.